Architecture Design

for

E-DBA

Version 1.0 approved

Prepared by Sibo Qiu 2230026131 Full
Yucheng Tong 2230026146 Full
Zheng Zhijinxiang 2230026222 Full
Manxi Tao 2230026145 Full
Yixin Xia 2230026171 Full
Boxin Su 2230026138 Full

A04

2025/3/25

Table of Contents

Table of Contents ..... **ii**
Revision History ..... **iii**
1. Overview **1**
1.1 Project description 1
1.2 References 1
1.3 Design purpose 1
2. Overall description **1**
Use case diagram and class diagram 1
2.1 Design model 3
2.2 System architecture 3
3. System architecture **4**
3.1 Payment 4
3.1.1 Description 5
3.1.2 Database 5
3.2 Workspace Management 6
3.2.1 Description 6
3.2.2 Database 6
3.3 User Authentication 7
3.3.1 Description 8
3.3.2 Database 8
3.4 Data & Service 9
3.4.1 Description 9
3.4.2 Database 10
4. Assessment **10**
4.1 Stability 10
4.2 Reusability 10
4.3 Scalability 11
5. Alternative design (optional) **11**
6. More considerations **11**
7. Appendix **11**

Revision History

Name	Date	Reason For Changes	Version
Manxi Tao	2025/3/25	3.1 Payment subsystem	0.1
Sibo Qiu	2025/3/25	3.2 Workspace Manager subsystem	0.2
Zhijingxiang Zheng	2025/3/27	3.3 User Authentication	0.3
Sibo Qiu	2025/3/27	Changed class name Service to Service Status	0.4
Yucheng Tong	2025/3/27	3.4 Data & Service	0.5
Boxin Su	2025/3/27	Add other parts	1.0

1. Overview

1.1 Project description

More and more organizations need to share data with other organizations. Registering higher education institutions requires secure data sharing with other schools in a transparent and interoperable manner. This SRS specifies the software requirements for the Education Data Bay Area (E-DBA) system. The E-DBA system is designed to provide a secure and efficient platform for data sharing among registered higher education organizations. The scope of this SRS includes the functional and non-functional requirements of the entire system, covering features such as student identity authentication, thesis sharing, course information sharing, and an optional vault service. The SRS also outlines the system's integration with external payment gateways and interfaces for accessing public and private data.

1.2 References

[1] A04. (2025). SRS_a04_v2.6 (Version 2.6).
[2] A04. (2025). UI_diagram (Version 2.6).

1.3 Design purpose

This modular architecture is designed to organize complex functionalities (payments, authentication, workspace management, etc.) into independent, cohesive modules, ensuring clear separation of concerns while maintaining well-defined interfaces for interaction. The primary goals are to enhance maintainability (enabling isolated updates without system-wide impact), improve scalability (supporting seamless addition of new features), and optimize development efficiency (allowing parallel work and reuse of common components). By isolating sensitive operations (e.g., payments) and standardizing reusable functions (e.g., email services), the design reduces technical debt, mitigates risks of cascading failures, and accommodates future business rule changes with minimal disruption. This approach is particularly suited for systems requiring frequent updates, strict security boundaries, or anticipated growth, balancing immediate development agility with long-term stability.

2. Overall description

Use case diagram and class diagram

Use Case diagram:

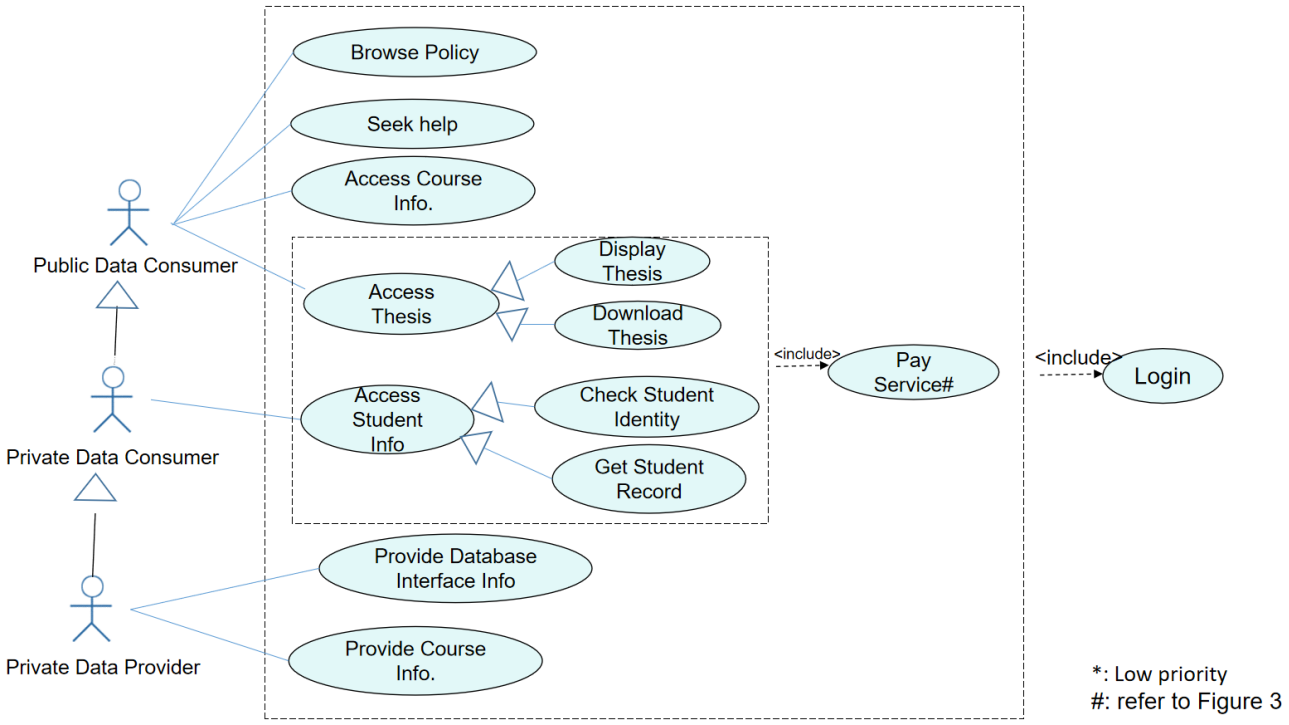


Figure 2

Figure 1

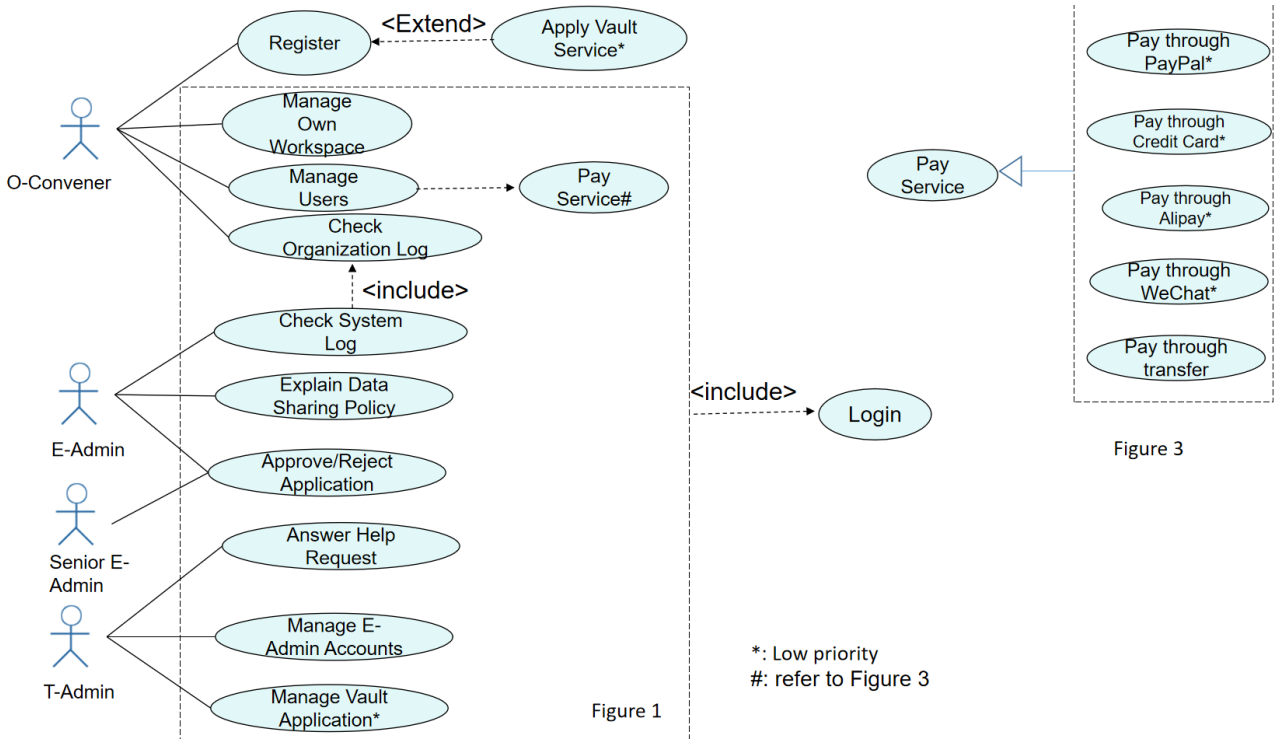


Figure 2

Class diagram:

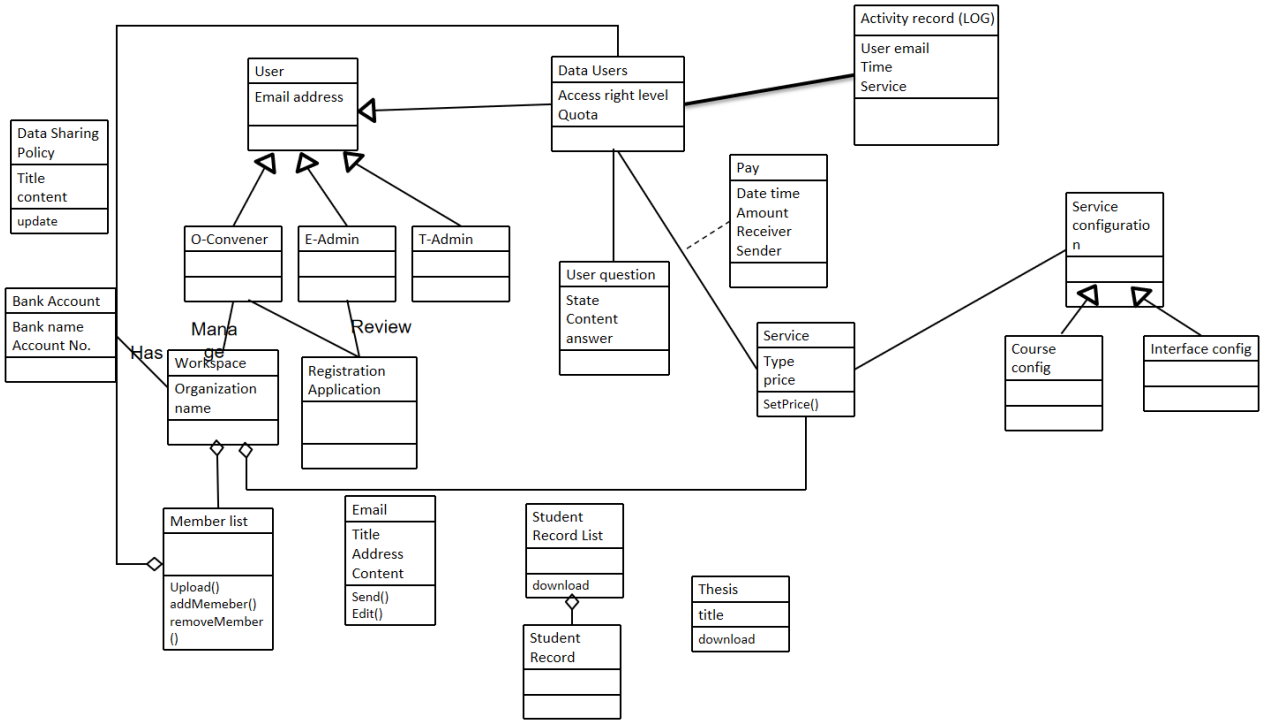


Figure 3

2.1 Design model

The system adopts a modular architecture, decomposing functionality into discrete subsystems, User Authentication, Workspace Management, and Payments—each dedicated to specific tasks. These modules interact through well-defined interfaces, enabling independent development, testing, and maintenance. This design ensures scalability, separation of concerns, reusability, and security. By organizing subsystems around core functions, the system balances flexibility with maintainability.

2.2 System architecture

This modular system architecture consists of four tightly integrated subsystems: The **User Authentication Subsystem** validates identities and shares user data with both the **Workspace Subsystem** (for configuration management) and **Data & Service Subsystem** (for centralized operations). The **Workspace Subsystem** dynamically interacts with the **Payment Subsystem** to process service-related transactions while coordinating with the **Data & Service Subsystem** to maintain consistent configurations. The **Data & Service Subsystem** serves as the backbone, providing shared resources and service mappings to all other components, while the **Payment Subsystem** completes the ecosystem by handling transactions after verifying user permissions through the authentication layer.

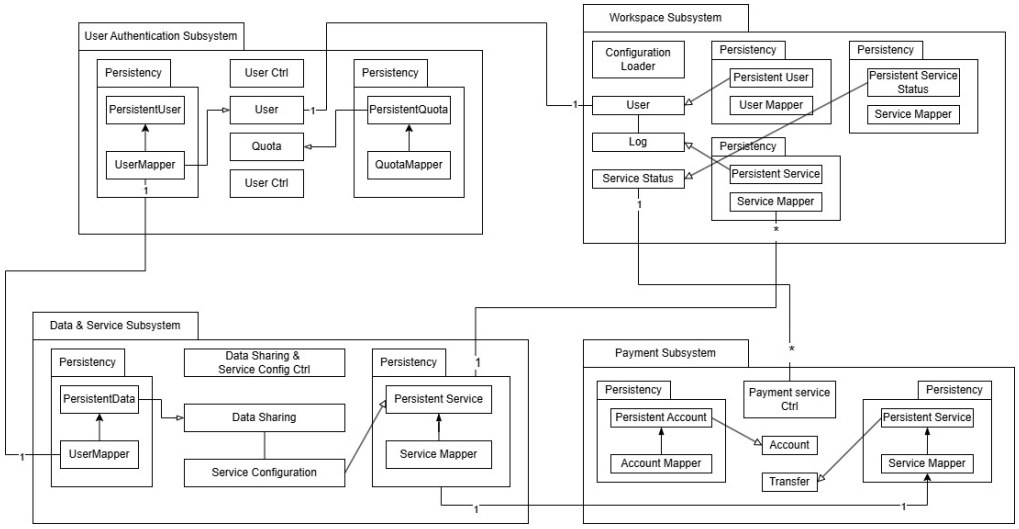


Figure 4

3. System architecture

3.1 Payment

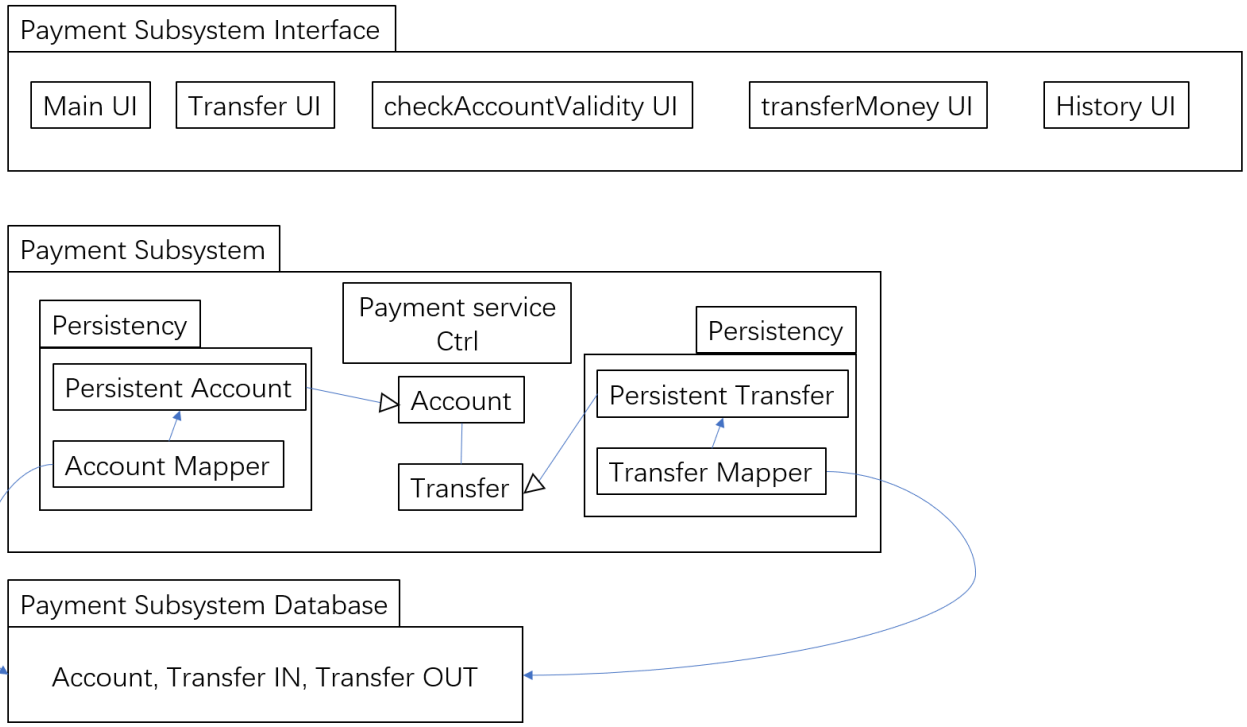


Figure 5

The components of the Payment subsystem include Payment Subsystem Interface， Payment Subsystem and Payment Subsystem Database three parts. UI part includes Main UI, Transfer UI,

checkAccountValidity UI, transferMoney UI, History UI. In Payment Subsystem part include Persistent Account, Account, Payment service Ctrl, Account Mapper, Persistency, Transfer, Persistent Transfer, and Transfer Mapper.

3.1.1 Description

The Main UI is the primary interface for users to interact with the payment subsystem, providing an entry point for various payment - related operations. The Transfer UI is specifically designed for handling transfer - related interactions, allowing users to input transfer - related information. The checkAccountValidity UI is responsible for verifying the validity of accounts, which is crucial for ensuring the security of payment operations. The transferMoney UI is used to execute the money - transfer action.

The Payment Subsystem Interface acts as a bridge between the subsystem and external systems, coordinating interactions and data exchanges. Persistent Account and Account are involved in handling account - related data, with Persistent Account responsible for the persistent storage of account information, and Account handling the account - related business logic. The Payment service is in charge of processing payment - related operations, and Ctrl plays a role in controlling and scheduling the overall process. The Account Mapper is responsible for mapping account - related data between different layer.

The History UI is used to display payment history records for users to query. The Transfer component is responsible for handling transfer - related business logic, Persistent Transfer is used for the persistent storage of transfer - related data, and the Transfer Mapper is used to map transfer - related data.

3.1.2 Database

Account			
Bank name	Account name	Account number	password

Transfer IN			
Bank name	Account name	Account number	password

Transfer OUT			
Bank name	Account name	Account number	Amount

Figure 6

Subsystem Database contains three tables: Account, Transfer IN, and Transfer OUT.

The Account table stores information such as bank name, account name, account number, and password. This information is essential for identifying and authenticating account holders. The Transfer IN table records information related to incoming transfers, including bank name, account name, account number, and password, which helps in tracking and processing incoming funds. The Transfer OUT table records information about outgoing transfers, including bank name, account name,

account number, and the transfer amount, enabling the system to manage and record transfer operations.

3.2 Workspace Management

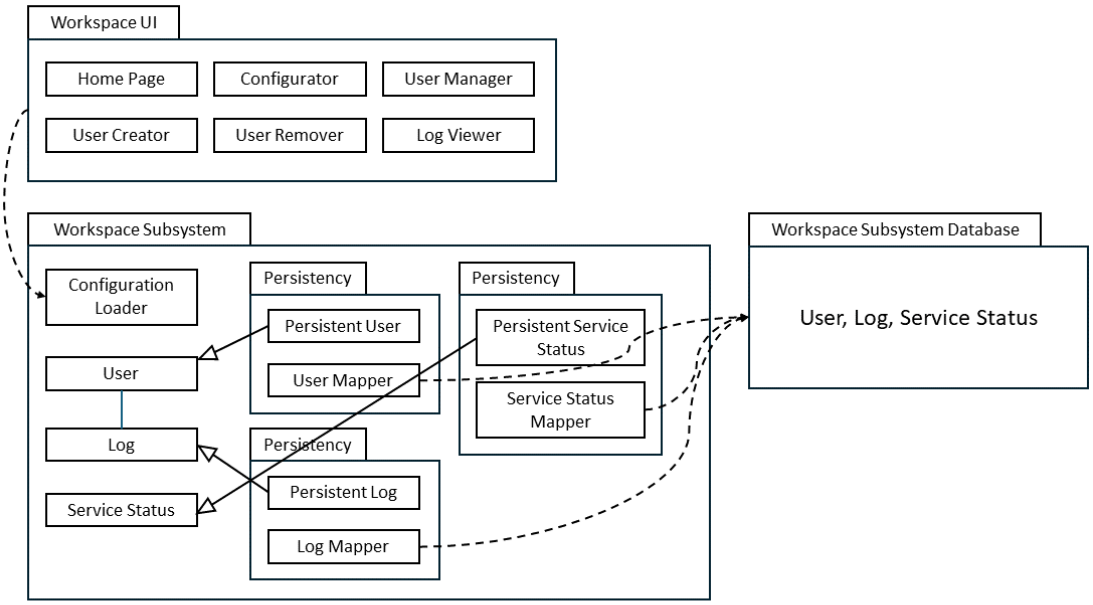


Figure 7

3.2.1 Description

Workspace Management subsystem consists of UI, main subsystem, and database. UIs remain consistent to their original designs in Requirement Specification. Main subsystem mainly consists of user management, log management, and service management, and all of them are stored persistently in databases.

Configuration Loader reads O-Convener's options in GUI, and execute operations, including user add/remove, show logs in Log Viewer, toggle services' availability.

3.2.2 Database

User				
userID	name	email	role	access_level

Log		
logID	userID	content

Service Status		
serviceID	name	availability

Figure 8

There are a total of 3 databases for Workspace Management subsystem: User, Log, and Service Status. User database keeps track of each user's information and access level within the system. Log database stores every user's activity. Service Status database stores which service(s) are provided to users in the organization.

3.3 User Authentication

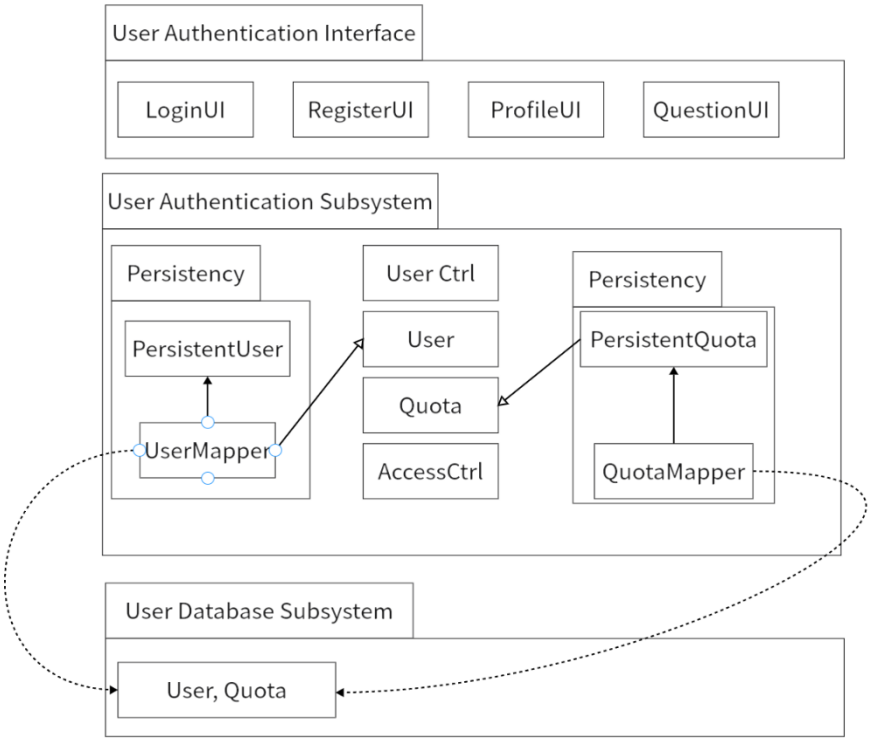


Figure 9

3.3.1 Description

This system follows a layered architecture where the User Authentication Interface (UI layer) handles the user-facing components like Login, Registration, Profile, and Security Questions. The User Authentication Subsystem (Control layer) manages logic through UserCtrl for user operations, AccessCtrl for ABAC enforcement, and Persistency to ensure data consistency and object mapping. The User Database Subsystem (Data layer) includes relational models (PersistentUser, PersistentQuota) and mappers for ORM and optimized queries, maintaining isolation and security. This design ensures clear separation of concerns, stateless control, data encapsulation, and robust security mechanisms across all layers.

3.3.2 Database

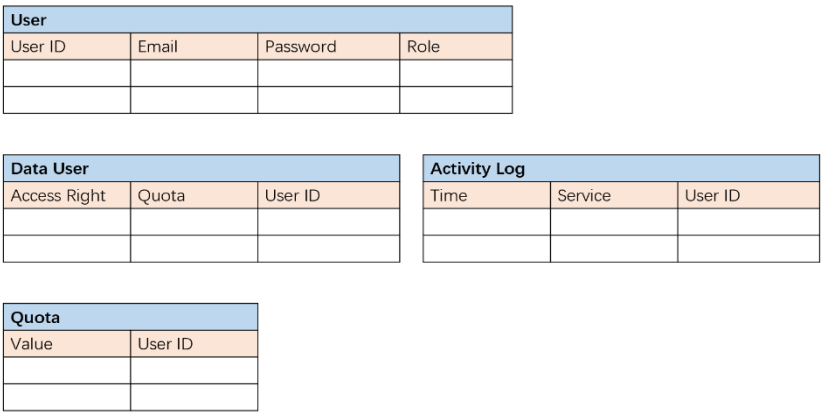


Figure 10

The database schema defines four normalized tables: User with unique emails and hashed passwords; DataUser linking users and quotas with role-based access; Activity Log recording immutable user actions with timestamps and service types; and Quota enforcing usage limits per user. Key features include referential integrity, auditability, role-based access, quota enforcement, and minimized redundancy via normalization.

3.4 Data & Service

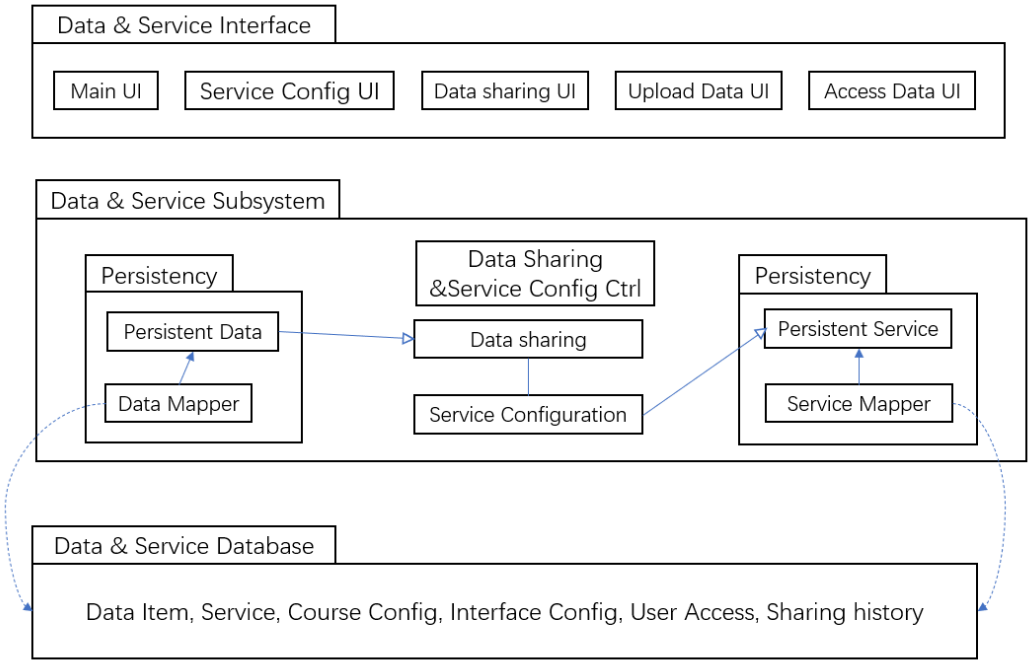


Figure 11

3.4.1 Description

The Data Sharing & Service Configuration Subsystem manages both data sharing and service configuration. It includes a user interface layer for interacting with users, a subsystem layer with controllers to handle business logic, and a persistence layer for storing data and configurations. The system ensures smooth operation by separating the interface, logic, and data storage while maintaining efficient data sharing and service management.

3.4.2 Database

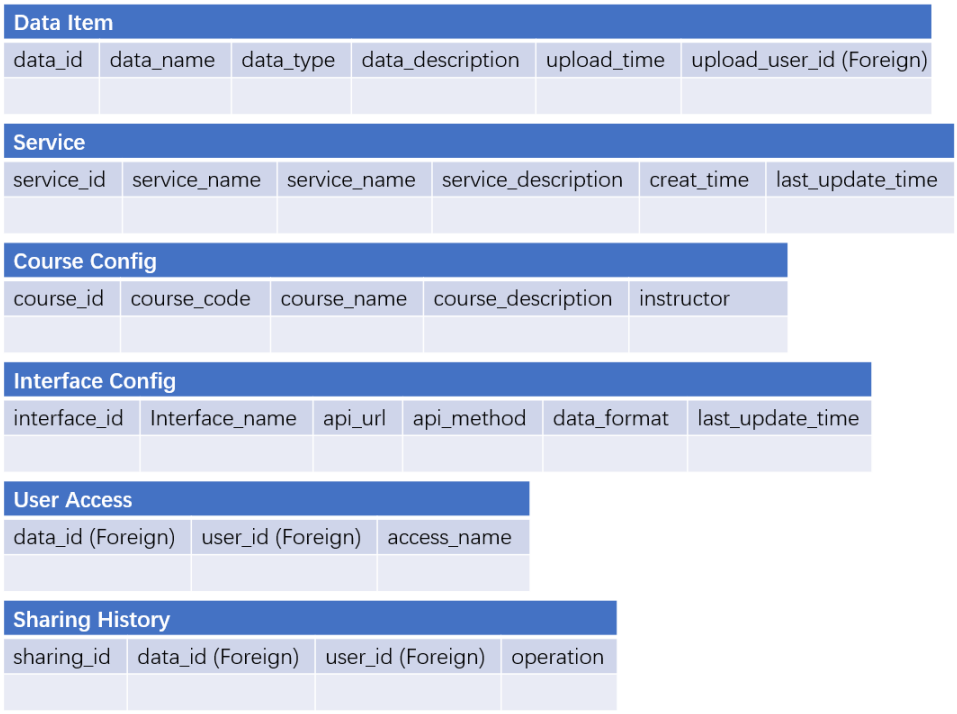


Figure 12

There are total of 6 databases for Data Sharing & Service Configuration Subsystem: Data Item, Service, Course Config, Interface Config, User Access, Sharing History. Key tables include Data Item for shared content, Service for configuration details, and User Access and Sharing History to track permissions and operations.

4. Assessment

4.1 Stability

The system demonstrates strong stability due to its modular architecture, which ensures low coupling between components while maintaining high cohesion within each module. Since each subsystem (e.g., Payments, User Authentication) operates independently through well-defined interfaces, modifications to one area (such as updating bank account handling) have minimal impact on other parts of the system.

4.2 Reusability

The architecture promotes excellent component reusability across the system. Common functionalities like email communication (Email.Send()) are designed as standalone features that can

be utilized by multiple subsystems without modification. The authentication components, including user roles and access controls, could potentially be repurposed for other systems with similar requirements.

4.3 Scalability

The modular design provides exceptional scalability for future growth and feature expansion. New capabilities can be added as independent modules without disrupting existing functionality - for example, introducing a notification subsystem or additional payment methods would require minimal changes to other components. The configuration-driven approach (seen in Interface Config and Course Config) allows for customization and extension without structural modifications. This flexibility ensures the system can evolve to meet changing requirements while maintaining stability and performance.

5. Alternative design

Null

6. More considerations

To further enhance the system's stability, reusability, and scalability, the following recommendations should be considered:

Stability Improvements:

Define Clear Interfaces: Ensure all inter-module communications (e.g., between Payments and Service) follow strict contracts (APIs or events) to prevent unintended dependencies.

Reusability Enhancements:

Standard Shared Components: Create a Common Utilities module for reusable functions (e.g., Email, Upload) to avoid duplication.

Scalability Strategies:

Adopt Event-Driven Architecture: For future expansions (e.g., notifications), use messaging queues to avoid tight coupling between modules.

7. Appendix

Null.